# The Y Language

A Formal Definition

the y compiler project

September 2026

**Abstract**

This document is the language definition of Y, a small statically-checked systems language that compiles directly to Linux x86-64 executables. It records the complete surface language, its type system, and its evaluation semantics — with particular precision for the mutating array operators (`+=`, `*=`), the increment/decrement and compound-assignment families, and their evaluated-exactly-once rules, so that these features are specified rather than discovered. Where the language is quirky, the quirk is written down; where a construct is reserved but unimplemented, it says so.

## Contents

**1 Overview** **2**

**2 Lexical structure** **3**
2.1 Source text . . . 3
2.2 Comments . . . 3
2.3 Identifiers and keywords . . . 3
2.4 Literals . . . 4
2.5 Operators and punctuation . . . 4

**3 Syntax** **5**
3.1 Structure and modules . . . 5
3.2 Declarations . . . 5
3.3 Types . . . 5
3.4 Statements . . . 6
3.5 Expressions . . . 6

**4 Types and static semantics** **7**
4.1 Scalar types . . . 7
4.2 Arrays . . . 7
4.3 Records . . . 7
4.4 Options and pointers . . . 7
4.5 Functions . . . 8
4.6 Annotations and inference . . . 8
4.7 Conversions . . . 8
4.8 Operator typing . . . 8
4.9 Compound assignment typing . . . 8
4.10 The poison law . . . 8

**5 Expressions and evaluation** **9**
5.1 Numeric semantics . . . 9
5.2 Logic . . . 9
5.3 Strings . . . 9
5.4 Assignment . . . 9
5.5 Increment, decrement and compound assignment . . . 9

**6 Arrays** **10**
6.1 Structure: length and capacity . . . 10
6.2 Operators . . . 11
6.3 Aliasing and visibility . . . 11
6.4 Fixed arrays . . . 12
6.5 Slices . . . 12
6.6 Byte strings . . . 12
6.7 Operators are the interface . . . 12

**7 Records and maps** **12**
7.1 The value model . . . 12
7.2 Records . . . 13
7.3 Maps . . . 13
7.4 Equality . . . 13
7.5 Copying . . . 13

**8 Functions, closures, options, pointers** **14**
8.1 Functions . . . 14
8.2 Tail calls . . . 14
8.3 Closures . . . 14
8.4 Options . . . 15
8.5 Pointers . . . 15

**9 Builtins** **15**
9.1 Sequences and text . . . 15
9.2 Conversions . . . 16
9.3 Binary data . . . 16
9.4 Options . . . 16
9.5 Input/output and the environment . . . 16
9.6 Pointers . . . 17

**10 Errors and diagnostics** **17**
10.1 Compile time . . . 17
10.2 Run time . . . 17
10.3 Exit status . . . 18
10.4 Reproducibility . . . 18

# 1 Overview

Y is a small imperative language with a static checker, designed so that a complete implementation of the language fits in one head: the compiler is roughly thirteen thousand lines of Y itself and compiles a program to a *static* x86-64 ELF executable using only its own assembler and ELF writer — no system assembler, linker, or C toolchain is involved at run time or at build time of user programs.

### Implementations

Two implementations exist and are kept in exact step:

- **the reference compiler**, written in Y (`src/y/`), self-hosting and byte-reproducible;
- **the C copy**, a line-for-line port of the same compiler in C (`src/c/`), sharing one value model. The two must produce byte-identical executables for every input.

This document defines the language both implement. Implementation detail is deliberately excluded except where it *is* observable behavior (value sharing, integer wraparound, trap conditions).

### Notation

Grammar productions are given in EBNF, with `typewriter` for terminals, *italic* for nonterminals, `[ ]` for option, `{ }` for repetition, `|` for alternation, and `( )` for grouping. Semantic rules are written as equational desugars or prose; the desugars are normative: an implementation may use any mechanism that produces the same observable behavior.

### A taste

```
// values, references, and the mutating operators
let xs = [1, 2, 3];      // heap array: len 3
xs += 4;                 // in-place push:        [1, 2, 3, 4]
xs *= 0;                 // in-place prepend:     [0, 1, 2, 3, 4]
let ys = xs + [9];       // NEW array, xs untouched

// the target of a compound assignment runs exactly once
def idx() -> long { print("called"); 2 };
a[idx()] += 10;          // idx() runs ONE time

// records are reference values too
let r = { name: "y", nums: [1, 2] };
r.nums += 3;             // visible through every binding of r
```

## 2 Lexical structure

### 2.1 Source text

A program is a sequence of bytes, conventionally UTF-8. The lexer works on bytes; only ASCII is significant to the grammar. Both `"\n"` (LF) and `"\r\n"` (CRLF) end a line. Positions reported in diagnostics are 1-based line and column, the column counted in bytes.

### 2.2 Comments

```
// a line comment runs to the end of the line
/* a block comment
   does not nest: the first */ closes it */
```

Comments are whitespace: they separate tokens and nothing more.

### 2.3 Identifiers and keywords

`isalpha`(c) is true for `A`–`Z`, `a`–`z` and `_`; `isdigit`(c) for `0`–`9`. An identifier is

*identifier* ::= *isalpha* { *isalnum* }

so `_` is a word character: `is_digit` is a single identifier, not three tokens. The prefix `y_` is *reserved*: identifiers beginning with `y_` are used by the compiler's generated code and must not be written by hand.

The keywords are:

```
let const def if else while for foreach in return true false null
```

## 2.4 Literals

**Integers.** Decimal, binary (`0b1010`) and hexadecimal (`0x1F`) radix literals are supported; `_` may separate digits in any position after the prefix (`1_000_000`). All integer literals are *long* (64-bit).

**Floats.** A digit sequence with a decimal point and/or an exponent: `1.5`, `1.5e+3`, `2.0e-1`. A dot followed by a non-digit is *not* a fractional part — `a..b` lexes as `a`, `..`, `b`. Float literals are IEEE-754 binary64 (*float*).

**Strings.** Double-quoted byte strings:

| escape | meaning |
|---|---|
| `\n \t \r` | LF, HT, CR |
| `"\" "\"" "\'"` | backslash, quote |
| `\xHH` | byte with hex value `HH` |
| `\c` (any other *c*) | *c*; the backslash is dropped |

A literal may not contain an unescaped newline or a NUL byte: string values are NUL-terminated byte strings and cannot carry NUL (use `[byte]` arrays, Section 9.3, for binary data).

**Byte strings.** `b"..."` and `b'...'` are *byte string literals*: a `[byte]` value spelled like a Python `bytes` literal. The escapes are the string escapes above, so binary data writes naturally: `b"\x7fELF"` is the four-byte ELF magic. Either quote spells the same type; single quotes exist only for byte strings.

**Booleans and null.** `true`, `false`, `null`.

## 2.5 Operators and punctuation

| kind | tokens |
|---|---|
| two-character | `== != <= >= && \|\| :: .. -> ++ - += -= *= /= %=` |
| single-character | `+ - * / % < > = ! &` |
| punctuation | `( ) { } [ ] , ; :` |

**Maximal munch.** The lexer always takes the longest operator that matches, exactly like C:

- `x+++y` is `x ++ + y`, not `x + ++y`;
- `x-y` is `x- y` (a parse error), not `x - -y`; write `x - -y` when negation is meant;
- `/=` is one token; `//` starts a comment, and `/ == y` must be written with a space to divide;
- there is **no** `=+` operator: `x =+ y` is `x = +y`. In-place prepend is `*=` (Section 6).

**Reserved but unimplemented.** The operator `::` lexes and parses but is rejected by the checker with "`operator '::' is unsupported`"; it is reserved for a future list-splicing form.

# 3 Syntax

## 3.1 Structure and modules

A *program* is a sequence of *statement*s executed in order. Source files may be combined with imports; an import splices the imported file's text in front of the importing file's own code, under a namespace alias:

```
import io from "./std/io.y";   // io.f(...) addresses defs of io.y
```

An `export { a, b }` clause, when present, must be the *last* statement of a file. Paths resolve relative to the importing file's directory.

## 3.2 Declarations

*declaration* ::= `let` *name* [`:` *type*] `=` *expr* `;`
| `const` *name* [`:` *type*] `=` *expr* `;`
| `def` *name* `(` *params* `)` [`->` *type*] *block*

*params* is a comma-separated list of *name* `:` *type*; parameter annotations are **mandatory**, return types are optional (inferred from the body, Section 4). A `const` declaration currently behaves exactly like `let`; the keyword records intent and is reserved for future immutability checking.

Function literals may also appear bound or anonymous: `let f = def (x: long) -> long { x };`.

## 3.3 Types

| | |
|---|---|
| *type* ::= `long` \| `byte` \| `bool` \| `float` \| `str` | |
| \| `[` *type* `]` | (heap array) |
| \| `[` *type* `;` *nat* `]` | (fixed array) |
| \| *type* `?` | (option) |
| \| `*` *type* | (pointer) |
| \| `(` [*type* {, *type*}] `)` `->` *type* | (function) |
| \| `{` [*name* `:` *type* {, *name* `:` *type*}] `}` | (record shape) |

A record shape is structural; the shape `{ k: str }` is written to mean "a record with at least a field `k` of type *str*" and is the conventional shape-hole annotation for AST-like data. `nat` is a non-negative integer literal. The type name `int` is accepted as a synonym of `long` in annotations.

### 3.4 Statements

*statement* ::= *declaration*
| *expr* `;`
| *assignment* `;`
| `return` [*expr*] `;`
| *block*
| `if (` *expr* `)` *block* [`else` (*block* | *statement*)]
| `while (` *expr* `)` *block*
| `for (` *stmt* `;` *expr* `;` *stmt* `)` *block*
| `for` *name* `in` *expr* `..` *expr* *block*
| `foreach (` *name* `in` *expr* `)` *block*
| *block*

*block* is `{` *statement*$^*$ [*expr*] `}`: a block is an *expression* whose value is its trailing expression (if any). A block must never be empty; write `{ 0; }` where a placeholder is needed. `foreach` iterates the elements of an array; the `for` range form iterates `lo..hi` with the low bound inclusive and the high bound exclusive.

### 3.5 Expressions

*primary* ::= *long* | *float* | *string* | `true` | `false` | `null` | *name*
| `(` *expr* `)` | *block*
| `[` [*expr* {`,` *expr*}] `]` | `[` *expr* `;` *expr* `]`
| `{` [*name* `:` *expr* {`,`...}] `}` `{` [*string* `:` *expr* {`,`...}] `}`
| `def (` *params* `)` [`->` *type*] *block*

*postfix* ::= *primary* { `(` [*args*] `)` | `[` *expr* `]` | `[` *expr* `?:` *expr* `?` `]` | `.` *name* | `++` | `--` }
*unary* ::= [`!` | `-` | `*` | `&` | `++` | `--`] *postfix*

#### Operator precedence

All binary operators are left-associative; lower rows bind tighter:

| level | operators | notes |
|---|---|---|
| 1 | `\|\|` | short-circuit |
| 2 | `&&` | short-circuit |
| 3 | `==` `!=` | structural equality |
| 4 | `<` `>` `<=` `>=` | numeric, or lexicographic *str* |
| 5 | `::` | one-element cons (Section 6) |
| 6 | `+` `-` | numeric, string/array `+` |
| 7 | `*` `/` `%` | numeric |

### Assignment forms

*assignment* ::= *place* `=` *expr*
| *place* *compound-op* *expr* `+= -= *= /= %=`
| *place* `++` | `++` *place* | *place* `--` | `--` *place*

A *place* is a name, an index expression `a[e]` (arrays and maps), or a field expression `o.f` (records), possibly nested: `grid[0][1]`, `rec.v[2]`, `m['k'][i]`. Field access is symbolic and exclusive: `r.f` for records, `m['k']` for maps. Pointers add `*p = e`. Section 5.5 defines the exact evaluation of every form; all of them evaluate the place's subexpressions **exactly once**.

# 4 Types and static semantics

Checking is static and *fatal*: a type error is a compile error. The type system is small — scalars, arrays, records, options, pointers and functions — and mostly structural.

## 4.1 Scalar types

| type | domain | notes |
|---|---|---|
| `long` | 64-bit signed integers | wraparound on overflow |
| `byte` | 0 . . . 255 | stored in one byte inside `[byte]` |
| `bool` | `true`, `false` | |
| `float` | IEEE-754 binary64 | |
| `str` | immutable byte strings | no embedded NUL |

## 4.2 Arrays

`[T]` is a heap array of *T*, `[T; N]` a fixed-length array of *T* with *N* a literal. The element type is part of the type and arrays are **homogeneous**: every element of an array has exactly the element type. Heterogeneous array literals are rejected:

```
let bad = [1, "two"];        // error: array element: expected long, got str
```

This applies recursively — `[[1], [2]]` has type `[[long]]`, and every operator that mixes arrays and elements (Section 6) enforces the same rule.

## 4.3 Records

A record literal `{ a: 1, b: "x" }` has the structural shape `{ a: long, b: str }`; field order is preserved and is part of the type. A field read is legal when the static shape has the field. The shape-hole form `{ k: str }` matches any record with at least those fields and is used where the precise shape is not known statically (it poisons quietly rather than erroring — see Section 4.10).

## 4.4 Options and pointers

`T?` is "a *T* or `null`"; `null` is its empty value and `some`/`unwrap`/`is_some` (Section 8.4) manipulate it. `*T` is a raw machine pointer; `&e` yields one (Section 8.5). Options are *not* sum types: any value may be compared against `null`.

### 4.5 Functions

`(T, T, ...) -> R`. Parameter types are mandatory; return types are inferred when omitted and checked when written. Function values are first-class (they may be bound, passed, stored and called); see Section 8.

### 4.6 Annotations and inference

- `def` parameter annotations are **required**; the checker reports a missing one exactly once (Section 4.10).
- Return types are inferred from the body when omitted; when written, every `return` and the body's trailing value must conform.
- `let` annotations are optional and checked; an annotated *array literal adopts* the annotated element type: `let b: [byte] = [104, 105];` packs those elements one byte each. Adoption is restricted to literals — assigning an existing `[long]` to a `[byte]` binding is a type error (the layouts differ).
- An empty array literal `[]` or empty record `{}` needs an annotation (there is nothing to infer from).

### 4.7 Conversions

Exactly three implicit conversions exist: `long` *to* `float` (the only numeric widening), `byte` *to* `long`, and `byte` *to* `float`. An annotated `byte` position accepts a `long` literal and masks it to eight bits where it is stored. There is no cast syntax.

### 4.8 Operator typing

Arithmetic requires numbers; if either side is `float` the result is `float` and the other side widens. `+` additionally allows strings (concatenation, with auto-stringification of the non-string side) and arrays (Section 6); arrays win over strings when both appear. Comparisons yield `bool`; `<...>=` on two strings is lexicographic byte order. `&&`/`||` require booleans. Equality works on any two values of the same type (structural for arrays and records).

### 4.9 Compound assignment typing

For a target of type $T$ and value of type $U$:

| $T$ | operators | rule |
|---|---|---|
| numbers | `+= -= *= /= %=` | $U = T$ (or widening) |
| `str` | `+=` only | $U = str$ |
| array | `+=`, `*=` only | Section 6 |

Anything else is a compile error, e.g. "`arrays support only '+=' and '*=' (in place), not '-='`". `++`/`--` require a numeric target.

### 4.10 The poison law

Missing information (a missing annotation, an unanalyzable expression) produces an *unknown* type that unifies with everything. The law this enables: **the root cause is reported exactly once** — an unknown-typed piece never generates follow-on errors at every enclosing use. Concretely, unknown unifies silently in element checks, call arguments and branches; the missing annotation itself is the single diagnostic.

# 5 Expressions and evaluation

Evaluation is strict and left-to-right; every operand is evaluated exactly once per execution of the enclosing expression. Side effects include calls, reads of places with side-effecting subexpressions, and the mutating array operators.

## 5.1 Numeric semantics

`long` arithmetic is two's complement modulo $2^{64}$: overflow *wraps*, it does not trap. Division and remainder truncate toward zero (C semantics) and are subject to:

- division or remainder by zero is a runtime error (Section 10);
- `LONG_MIN / -1` evaluates to `LONG_MIN` (the quotient does not fit and would fault on x86) and `LONG_MIN % -1` evaluates to `0`. Implementations must special-case these two.

`float` arithmetic is IEEE-754 binary64. Mixed long/float operands widen the long side.

## 5.2 Logic

`&&` and `||` short-circuit: the right operand is evaluated only when the left does not decide the result. Both operands must be `bool`.

## 5.3 Strings

Strings are immutable values: concatenation `a + b` produces a new string. If exactly one side of `+` is a string, the other side is stringified (Section 9.2) and concatenated. Comparisons `< <= > >=` on two strings compare bytes lexicographically; `==` compares contents.

## 5.4 Assignment

`p = e` evaluates `e` and stores it into the place `p`. Assignment is an expression yielding the assigned value. Section 7 defines what storing into arrays, records and variables means for aliasing.

## 5.5 Increment, decrement and compound assignment

The four increment/decrement forms and the five compound-assignment operators share one rule:

> **Exactly-once rule.** The target place's subexpressions are evaluated *exactly once*. Writing `a[f()] += e` evaluates `f()` one time, no matter how the place is read and written.

Formally, let a *place* be $p ::= x \mid a[i] \mid o.f$, possibly nested. Define $\mathrm{cap}(p)$, the capture of a place into fresh variables (every non-name subexpression runs once):

$$\begin{aligned} \mathrm{cap}(x) &= \epsilon \\ \mathrm{cap}(a[i]) &= \mathrm{cap}(a); \ \text{ if } a \text{ is not a name: } \texttt{let } \beta = a\texttt{; let } \iota = i\texttt{;} \\ \mathrm{cap}(o.f) &= \mathrm{cap}(o); \ \text{ if } o \text{ is not a name: } \texttt{let } \beta = o\texttt{;} \end{aligned}$$

with $\mathrm{read}(p)$ and $\mathrm{write}(p, v)$ the corresponding read and store through the captured names. Then the *numeric* forms desugar as:

$$
\begin{aligned}
\texttt{p++} &\equiv \text{cap}(p);\ \{\ \texttt{let t = read}(p);\ \texttt{write}(p, \texttt{t + 1});\ \texttt{t}\ \} \\
\texttt{++p} &\equiv \text{cap}(p);\ \{\ \texttt{let t = read}(p)\ \texttt{+ 1};\ \texttt{write}(p, \texttt{t});\ \texttt{t}\ \} \\
\texttt{p-} &\equiv \text{cap}(p);\ \{\ \texttt{let t = read}(p);\ \texttt{write}(p, \texttt{t - 1});\ \texttt{t}\ \} \\
\texttt{-p} &\equiv \text{cap}(p);\ \{\ \texttt{let t = read}(p)\ \texttt{- 1};\ \texttt{write}(p, \texttt{t});\ \texttt{t}\ \} \\
\texttt{p op= e} &\equiv \text{cap}(p);\ \{\ \texttt{let t = read}(p)\ \textit{op}\ e;\ \texttt{write}(p, \texttt{t});\ \texttt{t}\ \}
\end{aligned}
$$

for $\textit{op} \in \{+, -, \times, \div, \%\}$ on numbers. Each form is an expression: postfix yields the *old* value, prefix the *new* one, and `p op= e` the new value of the place.

**Examples.**

```
a[f()]++;          // { let y_t = f(); let v = a[y_t];
                   //   a[y_t] = v + 1; v }   -- f() runs once
rec.v[g()] += 1;   // g() once; the read and the write use the same element
x.y.z += e;        // x and x.y evaluated once
```

**Arrays.** On array targets `+=` and `*=` are *primitive in-place* operations — they do not desugar through `+` (which copies) — and are defined in Section 6. On string targets only `+=` is legal and concatenates. Every other combination is a type error (Section 4.9).

# 6 Arrays

Arrays are the language's one growable sequence. They are **reference values**: a binding names an array *object*, and mutating operators write into that object so every binding sees the change. Arrays are **homogeneous**: one element type, enforced everywhere.

## 6.1 Structure: length and capacity

An array object carries two independent quantities:

| | meaning | invariant |
|---|---|---|
| `len` | number of live elements | $\texttt{len} \geq 0$ |
| `cap` | elements the storage can hold | $\texttt{len} \leq \texttt{cap}$ |

Indexing is trapped: reading or writing `a[i]` with `i < 0` or `i >= len` is a runtime error ("index out of bounds"). `cap` is never observable through operators — only `len` is (`len(a)`).

**Growth policy (amortized doubling).**

$$
\begin{aligned}
\texttt{cap}_0 &= 0 \\
\texttt{cap}_{k+1} &= \max(8,\ 2 \cdot \texttt{cap}_k) \qquad \text{when } \texttt{len} \text{ reaches } \texttt{cap}
\end{aligned}
$$

That is: the array grows *only* when the capacity is reached; the first growth yields capacity 8 and every later growth doubles it $(8, 16, 32, \ldots)$. Appending $n$ elements one at a time therefore costs $O(n)$ copies total, $O(1)$ amortized per element. The capacity never shrinks.

## 6.2 Operators

Let $a$, $b$ be arrays of $T$, and $x$ a value of type $T$. The complete operator semantics:

| expression | result | effect on operands | value types |
|---|---|---|---|
| `a + b` | new array $= a +\!\!+ b$ | none | $T = T$ |
| `a + x` | new array $= a +\!\!+ [x]$ | none | type of $x$ is $T$ |
| `x + a` | new array $= [x] +\!\!+ a$ | none | type of $x$ is $T$ |
| `x :: a` | new array $= [x] +\!\!+ a$ | none | $x$ is ONE element |
| `a += b` | $a$ | appends every element of $b$, in place | $T = T$ |
| `a += x` | $a$ | pushes $x$, in place | type of $x$ is $T$ |
| `a *= x` | $a$ | prepends $x$, in place | type of $x$ is $T$ |
| `a = b` | $b$ | aliases: both names name one array | |

In prose, matching the design intent exactly:

- `arr1 + arr2` — make a new array which is `arr1` concatenated with `arr2`; the operands are untouched.
- `arr1 += arr2` — append `arr2` to the end of `arr1`, *in place*.
- `arr1 = arr2` — both point to the same array in memory.
- `arr1 + elem` — a new array which is `arr1` concatenated with `[elem]`; only allowed if the element types match.
- `elem + arr1` — a new array which is `[elem]` concatenated with `arr1`.
- `+=` means in-place push (of an element) or append (of an array); `+` always means a *new* array; `*=` means prepend the item in place.

**The one-thing rule.** Whether `+=` pushes or appends is decided by the *shape of the value*: an array value appends its elements; any other value pushes as one element. Consequently, to push an array *as one element* of an array-of-arrays, wrap it in a one-element array literal:

```
let rows = [[1], [2]];
rows += [[3]];        // appends the single row [3]
rows += [[4]];        // and the row [4] -- one element each
```

**Homogeneity.** Every mixing form requires matching element types; the checker rejects mismatches — including the append case, where a non-matching array would splice wrong-typed elements:

```
let rows = [[1], [2]];
rows += [3];   // error: '+=' appended elements: expected [long], got long
```

**`a += a`.** The right side is captured (its length snapshotted) before appending, so `a += a` doubles `a` with a copy of its original elements.

## 6.3 Aliasing and visibility

Assigning arrays aliases; mutating operators are visible through every alias:

```
let m1 = [1, 2];
let m2 = m1;      // ONE array, two names
m1 += 3;          // m2 sees it: len(m2) == 3
```

By contrast `+` never mutates: `let ys = xs + [9];` leaves `xs` at its old length.

### 6.4 Fixed arrays

`[T; N]` arrays are allocated with exactly `N` slots; their length is part of the type, so they reject `push`/`+=`/`*=` growth. Indexing is bounds-trapped like heap arrays. A fixed array may be sliced; the result is a heap `[T]`.

### 6.5 Slices

`a[lo:hi]` evaluates to a **new** heap `[T]` holding the elements in [`lo`, `hi`). Missing bounds default to `0` and `len`; bounds are clamped (no trap). All four spellings are legal: `a[:]`, `a[lo:]`, `a[:hi]`, `a[lo:hi]`.

### 6.6 Byte strings

A `[byte]` array *is* a binary string: `b"..."` literals produce them, `+` concatenates, slices cut, and printing one shows the Python-style repr — printable ASCII verbatim, quote and backslash escaped, `\n` `\r` `\t` as escapes, and every other byte as `\xHH`:

```
print(b"\bs x7fELF");      // b'\bs x7fELF'
let hdr = b"AB" + b"\bs x00";
print(hdr);               // b'AB\bs x00'
print(hdr[0:2]);          // b'AB'
```

`\std/struct.y` builds on this with pack/unpack helpers for file headers and binary layouts (`u32be`, `get_i16le`, `pack`, `unpack`, ...).

### 6.7 Operators are the interface

There are no array mutator functions: `+=` and `*=` *are* push and prepend. `len(a)` is the only array builtin.

## 7 Records and maps

Y has *two* key/value types, deliberately distinct:

| | **record** (static) | **map** (dynamic) |
|---|---|---|
| type | `{ a: long, b: str }` | `{str: long}` |
| keys | identifiers, fixed at compile time | any hashable value at run time |
| values | may differ per field | one homogeneous type $V$ |
| layout | C struct: fields at fixed offsets | growable key/value store |
| keying | `r.a` | `m['a']` |

### 7.1 The value model

| category | types | assignment |
|---|---|---|
| values | `long byte bool float str null` | copies the bits |
| references | arrays (`[T]`, `[T; N]`), records, maps | **aliases** the object |

"Reference" means: every binding of the same object names one storage, and every in-place mutation (`+=`, `*=`, field assignment, map stores) is observable through all of them.

## 7.2 Records

A record literal `{ name: "y", n: 1 }` allocates one record object whose shape is the literal's fields *as written*; field order is preserved. At run time a record is exactly a C struct: field *i* is the word at `+8i` from the record address, so `r.a` compiles to a single load at a compile-time offset.

```
let r = { v: [7], n: 1 };
let s = r;              // aliases the record
r.v += 8;               // s.v sees it
s.n = 2;                // r.n sees it
```

The shape is fixed: writing a field the shape does not have is a *compile error* (there is no runtime field growth), and a field read requires the field to exist statically.

**Record spread.** `{a: 0, ...obj}` builds a record whose shape is the *union* of the two shapes entirely statically; no dynamic shape is involved. Entries merge with last-wins values and first-appearance positions, so `obj` may overwrite `a` and a later explicit key may overwrite what `obj` spread:

```
let b = {x: 1, y: 2};
let c = {y: 9, z: 3};
print({a: 0, ...b});      // {a: 0, x: 1, y: 2}
print({...b, ...c});      // {x: 1, y: 9, z: 3}
print({...b, y: 100});    // {x: 1, y: 100}
```

## 7.3 Maps

A map type is `{K: V}` where *K* is a hashable key type (`str`, `long`, `byte`, `bool`, `float`) and *V* is one homogeneous value type. Map literals use quoted keys (identifier keys are record syntax):

```
let m = {"a": 1, "b": 2};       // {str: long}
let empty: {str: long} = {};    // empty literals need the annotation
m["c"] = 3;                     // insert or overwrite
print(m["a"]);                  // 1
print(m["zz"]);                 // null -- reads yield V?
```

Keying is symbolic and exclusive: `m['k']` is the map form and is reserved for maps; `r.a` is the record form. Reading `m[k]` yields *V*`?` `null` when the key is absent and `m[k] = v` stores in place (the store is visible through every binding of the map). `len(m)` is the number of entries. The map grows as needed (amortized doubling, like arrays).

## 7.4 Equality

`==` on two arrays or two records is **structural**: same length/fields, pairwise-equal contents. It is never a pointer comparison; `arr == []` is an emptiness test.

## 7.5 Copying

There is no implicit deep copy. To duplicate an array use `+` with an empty right side (`a + []`) or slicing (`a[:]`); records can be rebuilt field by field. The language deliberately makes sharing the default and copying explicit.

# 8 Functions, closures, options, pointers

## 8.1 Functions

A `def` declares a function; parameters are typed, the return type is inferred or checked. Recursion uses `self` (a function's own name is *not* in scope inside its body — write `self(...)`):

```
def fact(n: long) -> long {
    if (n <= 1) { 1 } else { n * self(n - 1) }
};
```

Functions are first-class values: they may be bound (`let f = g;`), stored in arrays and records, passed to other functions, and called through any of those (`f(1)`, `fs[0](1)`, `r.handler(1)`). Calling a non-function is a compile error.

## 8.2 Tail calls

A self-call in *tail position* — the body's trailing expression, a `return`'s value, or an if-arm's trailing expression — recycles the stack frame instead of growing it: the arguments are evaluated, the frame is torn down, and control re-enters the function. Tail recursion therefore runs in constant stack space:

```
def sum_to(n: long, acc: long) -> long {
    if (n == 0) { acc } else { self(n - 1, acc + n) }
};
```

Calls outside tail position (`self(x) + self(y)`, `let t = self(x)`, inside loops) are ordinary calls and stack as usual. Instrumented builds (`-prof`) keep plain calls so every profiled entry pairs with its exit.

## 8.3 Closures

A function literal closes over its *entire lexical scope*, not over the individual variables it mentions. Captured bindings are **shared**: a write inside the closure is visible outside, and a write outside is visible inside.

```
def counter() {
    let n = 0;
    def tick() -> long { n = n + 1; n };
    tick
};
let t = counter();
t(); t();
print(t());        // 3 -- the one shared n
```

Each *invocation* of the enclosing function creates fresh bindings, so closures created in different iterations see different cells ("`let` semantics"):

```
while (i < 3) {
    let x = i;
    fns = push(fns, def() -> long { x });
    i = i + 1;
};
```

### 8.4 Options

`T?` is the type of "a *T* or `null`". `some(v)` boxes a value as present, `is_some(v)` tests, `unwrap(v)` returns the payload and is a *runtime error* on `null` (it is a programmer error, loudly). Comparisons with `null` work directly; an option of an array is a nullable reference, not a copied one.

### 8.5 Pointers

`*T` is a raw address of one *T*; `*p = v` stores, `*p` reads. The address-of operator `&e` is deliberately narrow:

- `&x` — address of a local or parameter;
- `&a[i]` — address of an array element (any array expression: `&grid[0][i]` is legal);
- `&arr` on a whole heap array, or `&` of any temporary, is a compile error (temporaries have no address).

Pointers are for interfacing with the system-call level (buffers for `read`/`write`-style calls); ordinary code uses arrays and records.

## 9 Builtins

The following functions are available without declaration. "Trap" marks a runtime error (Section 10).

### 9.1 Sequences and text

**`len(x)`** Length of a string (bytes), array (elements), record (fields) or map (entries) as `long`.

**`range(n)`** `[0, 1, ..., n-1]`.

**`substr(s, i, n)`**
Substring, C-clamped: `i < 0` becomes `0`, `i >= len` yields `""`, `n < 0` or overflow runs to the end.

**`trim(s)`** Strip leading/trailing space, tab, CR, LF.

**`join(xs, sep)`**
Concatenate `[str]` with a separator.

**`split(s, sep)`**
Split on every occurrence of a non-empty separator; returns `[str]` (empty for empty separator).

**`index_of(s, sub)`**
First index of `sub` in `s`, or `-1`.

**`last_index_of(s, sub)`**
Last index, or `-1`.

**`ord(s)`** First byte as `long`, or `-1` for empty.

**`chr(n)`** One-byte string of `n` (masked to 8 bits).

**`byte_at(s, i)`**
Byte at `i` as `long`; traps out of bounds. Cheap per-byte access (no copy).

### 9.2 Conversions

`tostring(v)`
: String form: numbers as decimal (floats with six decimals), `true`/`false`, strings unchanged, `null` as `"null"`, arrays as `"[e0, e1, ...]"`, records as `"{k: v, ...}"` with escaped keys.

`tonum(s)` Parse a decimal integer (`0` on garbage).

`typeof(v)`
: The static type as a printable value.

### 9.3 Binary data

`bytes(s)` `[byte]` of the string's raw bytes.

`from_bytes(b)`
: String from a `[byte]` array.

### 9.4 Options

`some(v)` A present option holding `v`.

`is_some(v)`
: `true` unless `v` is `null`.

`unwrap(v)`
: The payload; **traps** on `null`.

### 9.5 Input/output and the environment

`print(x)` Print a value followed by a newline to stdout.

`puts(s)` Write a string to stdout, no newline added.

`eputs(s)` Same, to stderr (keeps `-time` output and program output apart).

`readline()`
: One line from stdin without its newline, or `null` at end of input.

`read_file(path)`
: File contents as `str`, or `null`.

`write_file(path, s)`
: Write a string; `bool` success.

`write_bytes(path, b, mode)`
: Write `[byte]` binary-safe and chmod (`mode < 0` skips chmod); `bool` success.

`argv()` `[str]` of the program's arguments.

`exit(code)`
: Terminate with an exit code.

`clock_us()`
: Monotonic microseconds as `long`.

`heap_used()`
: Bytes allocated so far.

`sleep(ms)`
: Suspend the process.

`syscall(...)`
: Raw Linux syscall wrapper; the first argument is the syscall number.

`exec(cmd)`
: Run a shell command, return its exit code. Removed from the binary entirely by `-no-exec` (then calling it is a compile error).

### 9.6 Pointers

`ptr_addr(p)`
: The address of a pointer as `long`.

Mutating and keying are symbolic, not functional: arrays use `+=` / `*=`, records use `r.f`, maps use `m['k']` and `m['k'] = v`. The legacy functions `push`, `prepend`, `map_get` and `map_set` do not exist.

## 10 Errors and diagnostics

### 10.1 Compile time

Static checking is fatal: a rejected program compiles to nothing. Diagnostics are one line each,

```
line:col:  error:  message
```

positioned in the user's source even when the compilation splices prelude/runtime code in front of it. The checker reports a root cause *once* (Section 4.10): a missing annotation or unanalyzable expression poisons quietly instead of cascading errors through every enclosing use. Typical families:

- name errors: `undefined variable 'x'`;
- shape errors: `record has no field 'f'`;
- operator typing: `'+=' appended elements:  expected [long], got long`, `arrays support only '+` `'++' needs a number target`;
- call errors: `call takes 2 args, got 1`;
- non-value use: `cannot index long (only arrays)`.

### 10.2 Run time

The compiled program traps — writes a one-line message to stderr and exits with status 1 — on:

| condition | message |
|---|---|
| array index outside `[0, len)` | `runtime error:  index out of bounds` |
| division or remainder by zero | `runtime error:  division by zero` |
| `unwrap(null)` | `runtime error:  unwrap of null` |
| missing dynamic field/key | `runtime error:  record has no field 'k'` |
| memory exhausted | `out of memory` |
| unbounded recursion | a recursion-limit error, not a crash |

Fixed arrays trap exactly like heap arrays; slices never trap (they clamp). Integer overflow is *not* an error (Section 5).

### 10.3 Exit status

A successfully run program exits with the code it chose (`exit(code)` or falling off the end, which is `0`); a trapped program exits with `1`. The compiler itself exits `0` on success, `1` on a rejected input.

### 10.4 Reproducibility

The compiler is deterministic: the same input produces byte-identical output on every run and in either implementation. This is checked continuously by comparing the two compilers' output binaries byte for byte, and by self-compilation (a compiler built by the compiler is identical to the one that built it).